

Конспект «Резервное копирование»

1. Резервное копирование (backup)

Резервное копирование — процесс создания копии данных на носителе (жёстком диске, дискете и т. д.), предназначенном для восстановления данных в оригинальном или новом месте их расположения в случае их повреждения или разрушения.

Резервное копирование необходимо для возможности быстрого и недорогого восстановления БД в случае утери рабочей копии информации по какой-либо причине.

2. Восстановление данных

Восстановление данных — процесс восстановления базы **данных до исходного состояния** из имеющейся резервной копии.

В процессе обычно используются утилиты для восстановления, поставляемые в комплекте с СУБД.

3. Полное резервное копирование (Full backup)

Данный метод создает **полную копию** набора исходных данных.

В данном случае копируется весь каталог с файлами БД либо создается образ диска или системы, где находится база.

4. Инкрементные бэкапы (Incremental backup)

Инкрементное резервное копирование использует полную копию, как начальную точку.

Затем выполняется резервное копирование только блоков данных, **которые были изменены** с момента последнего резервного задания, с заданным периодом выполнения задания.

В зависимости от политики хранения резервных копий через определенный период создается новая полная копия для повторения цикла.

5. Дифференциальный бэкап (differential backup)

Дифференциальный бэкап — это тип резервного копирования файлов, при котором **копируются не все исходные файлы, а только новые и измененные** с момента создания предыдущей полной копии.

Он является чем-то средним между полным резервным копированием и инкрементальным.

Является накопительным — каждая следующая копия содержит все новые/измененные файлы с момента создания предыдущей полной резервной копии.

6. Холодное резервирование БД

Холодное резервное копирование выполняется **на уровне операционной системы** при **остановленной базе данных** — резервируются файлы, составляющие базу данных файлы данных, управляющие файлы, файл параметров.

7. Горячее резервирование БД

При горячем резервировании база данных **включена и открыта для потребителей**.

Копия базы данных приводится в согласованное состояние путем автоматического приложения к ней журналов резервирования по окончании копирования файлов данных.

8. Репликация

Идея репликации основана на том, что кроме «главного» сервера («Мастера») постоянно работают [ведомые сервера \(«слейвы»\)](#), которые получают инкрементные бэкапы с мастера в режиме реального времени.

Репликация — это в первую очередь инструмент для обеспечения высокой доступности и аварийного восстановления, а [НЕ замена резервному копированию](#).

9. Резервное копирование и восстановление PostgreSQL

Идея, стоящая за этим методом, заключается в генерации текстового файла с командами SQL, которые при выполнении на сервере пересоздадут базу данных в том же самом состоянии, в котором она была на момент выгрузки.

Альтернативной стратегией резервного копирования является непосредственное копирование файлов, в которых Postgres хранит содержимое базы данных.

В процессе работы Postgres ведёт [журнал предзаписи \(WAL\)](#). В этот журнал записываются все изменения, вносимые в файлы данных.

Если происходит крах, целостность СУБД может быть восстановлена в результате «воспроизведения» записей, зафиксированных после последней контрольной точки.

Однако наличие журнала делает возможным использование третьей стратегии копирования баз данных: можно сочетать резервное копирование на уровне файловой системы с копированием файлов WAL.

Если потребуется восстановить данные, мы можем восстановить копию файлов, а затем воспроизвести журнал из скопированных файлов WAL, и таким образом привести систему в нужное состояние.

10. Резервное копирование и восстановление MySQL

Базу данных MySQL можно скопировать, если временно выключить MySQL-сервер и просто скопировать файлы из папки [/var/lib/mysql/db/](#).

Если сервер не выключить, вероятна потеря и порча данных. Для больших нагруженных баз эта вероятность близка к 100%.

При первом запуске с «грязной» копией базы данных MySQL-сервер начнет процесс проверки всей базы, который может затянуться на часы.

В большинстве «живых» проектов регулярное выключение сервера БД на длительное время неприемлемо. Для решения этой проблемы применяется трюк, основанный на [снэпшотах](#) файловой системы.

Общая схема действий такова:

- блокируются все таблицы,
- сбрасывается файловый кэш БД,
- делается снэпшот файловой системы,
- разблокируются таблицы.

Далее файлы копируются из снэпшота, который затем уничтожается.

«Блокирующая» часть такого процесса занимает время порядка нескольких секунд, что уже допустимо.

В качестве расплаты на какое-то время, пока «жив» снэпшот, снижается производительность файловых операций, что в первую очередь бьет по скорости операций записи в базу.

Для того, чтобы считать в бэкап данные из production-базы, не обязательно дергать файлы. Можно выбрать данные запросом и сохранить их в текстовый файл.

Для этого используется SQL-команда [SELECT INTO OUTFILE](#) и парная ей — [LOAD DATA INFILE](#).

Выгрузка производится построчно (можно отобрать для сохранения только нужные строки, как в обычном SELECT). Структура таблиц нигде не указывается — об этом должен заботиться программист.

Он также должен позаботиться о включении команд [SELECT INTO OUTFILE](#) в транзакцию, если это необходимо для обеспечения целостности данных.

На практике [SELECT INTO OUTFILE](#) используется для частичного бэкапа очень больших таблиц, которые нельзя скопировать никаким другим образом.

11. Инкрементные бэкапы

Традиционно рекомендуют держать **10 бэкапов**: по одному на каждый день недели, а также бэкапы двухнедельной, месячной и квартальной давности — это позволит достаточно глубоко откатиться в случае порчи каких-либо данных.

Храниться бэкапы должны точно **не на том же диске**, что и живая база, и **не на том же сервере**.

Эти требования могут стать проблемой для больших баз.

Частично решить эту проблему позволяют **инкрементные бэкапы**, когда полный бэкап делается, например, только по воскресеньям, а в остальные дни пишутся только данные, добавленные или измененные за прошедшие сутки.

12. Репликация

Реплика — это полная копия базы, но это не резервная копия!

Идея репликации основана на том, что кроме «главного» сервера («Мастера») постоянно работают ведомые сервера MySQL («слейвы»), которые получают инкрементные бэкапы с мастера в режиме реального времени.

Таким образом, время отката уменьшается почти до сетевого лага. В случае краха Мастера можно оперативно назначить «новым Мастером» один из слейвов и перенаправить клиентов на него.

Кроме того, слейвы могут обрабатывать запросы на чтение данных (**SELECT-ы**); это можно использовать для выполнения каких-то расчетов или снижения нагрузки на мастера.

13. Мониторинг состояния бэкапов

Зачем мониторить бэкапы:

- Гарантировать **актуальность** бэкапов (последняя копия создана сегодня).
- Проверять **целостность** (файл не битый, БД восстанавливается).
- Контролировать **достаточность ресурсов** (место на диске, сеть).

Что отслеживать:

- **Время создания** бэкапа (например: **last_backup_time < 24h**).
- **Размер файла** (аномалии = сигнал: **backup_size < 10MB при норме 1 ГБ**).

- [Результат проверки целостности](#) (тестовое восстановление в песочнице).
- [Логи ошибок](#) (ошибки **mysqldump**, **pg_dump**, **xtrabackup**).

Инструменты мониторинга:

- Скрипты + Cron: Самописные проверки (например, через **md5sum**).
- Специализированные утилиты:
 - MySQL: **mysqlbackup --verify** (Enterprise), **pt-checksum** (Percona).
 - PostgreSQL: **pg_verifybackup**, **pgbackrest check**.
- Системы мониторинга:
 - Zabbix
 - Prometheus + Grafana